LAPORAN PRAKTIKUM

POSTTEST 2

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Nama (2509106021)

Kelas (A1 ‘25)

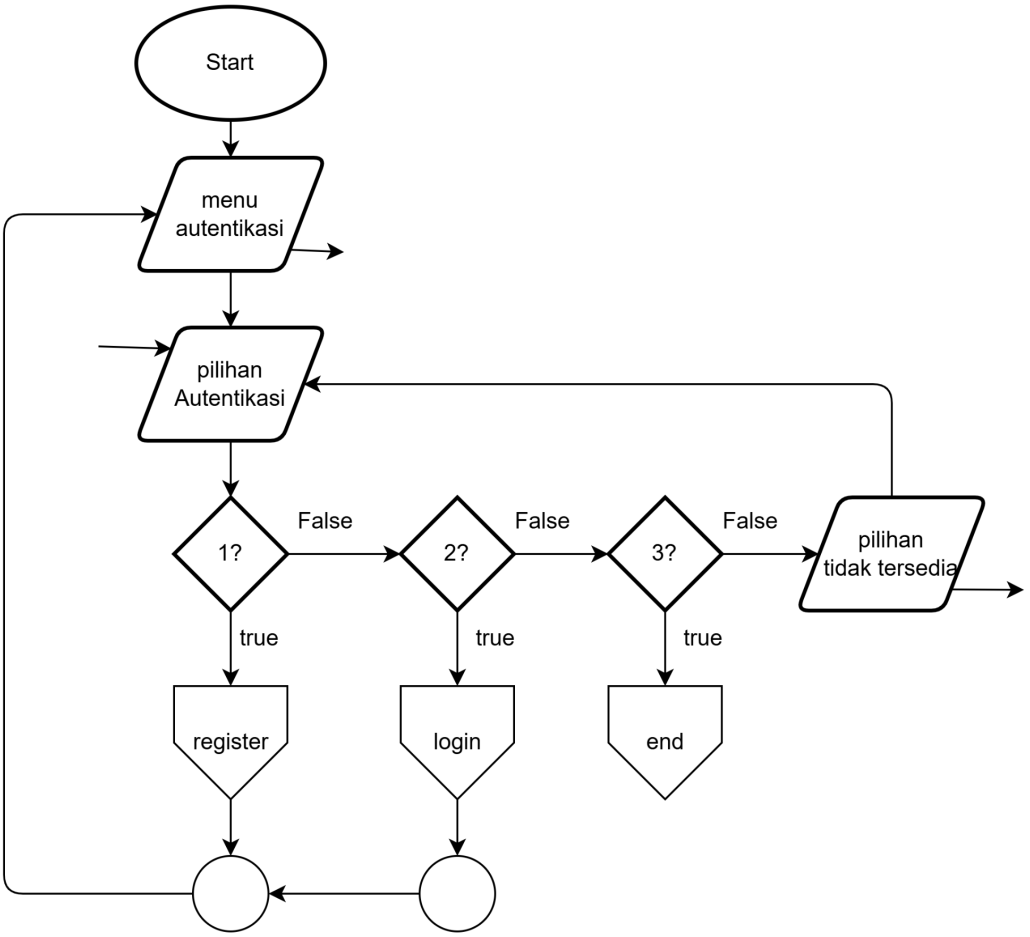
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

SAMARINDA

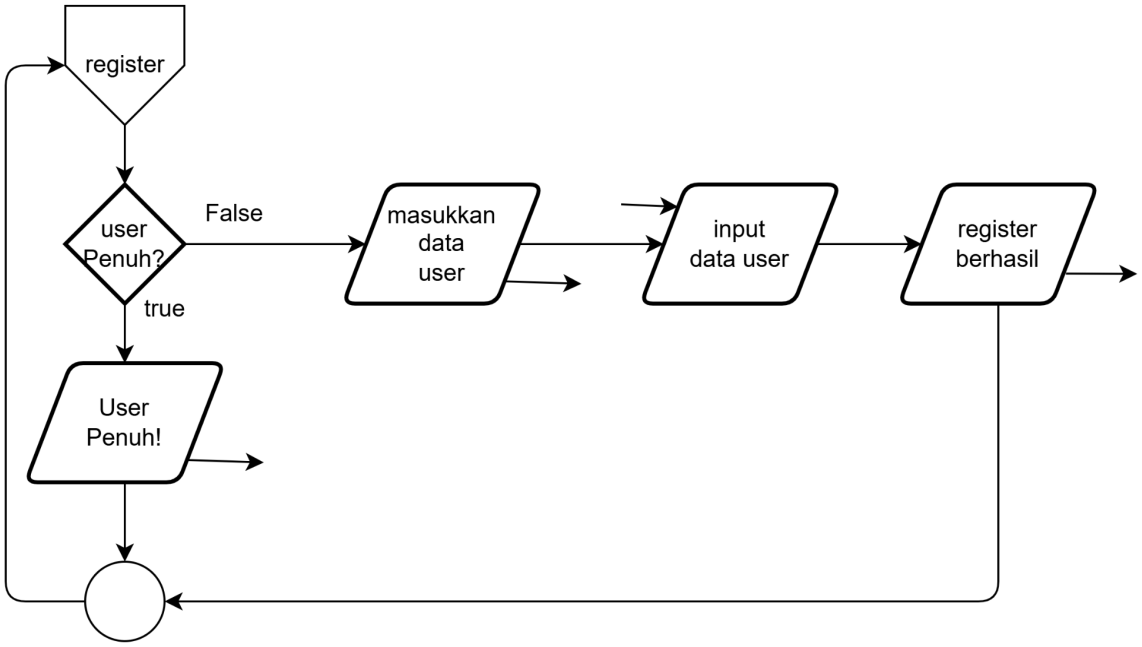
2025

1. Flowchart



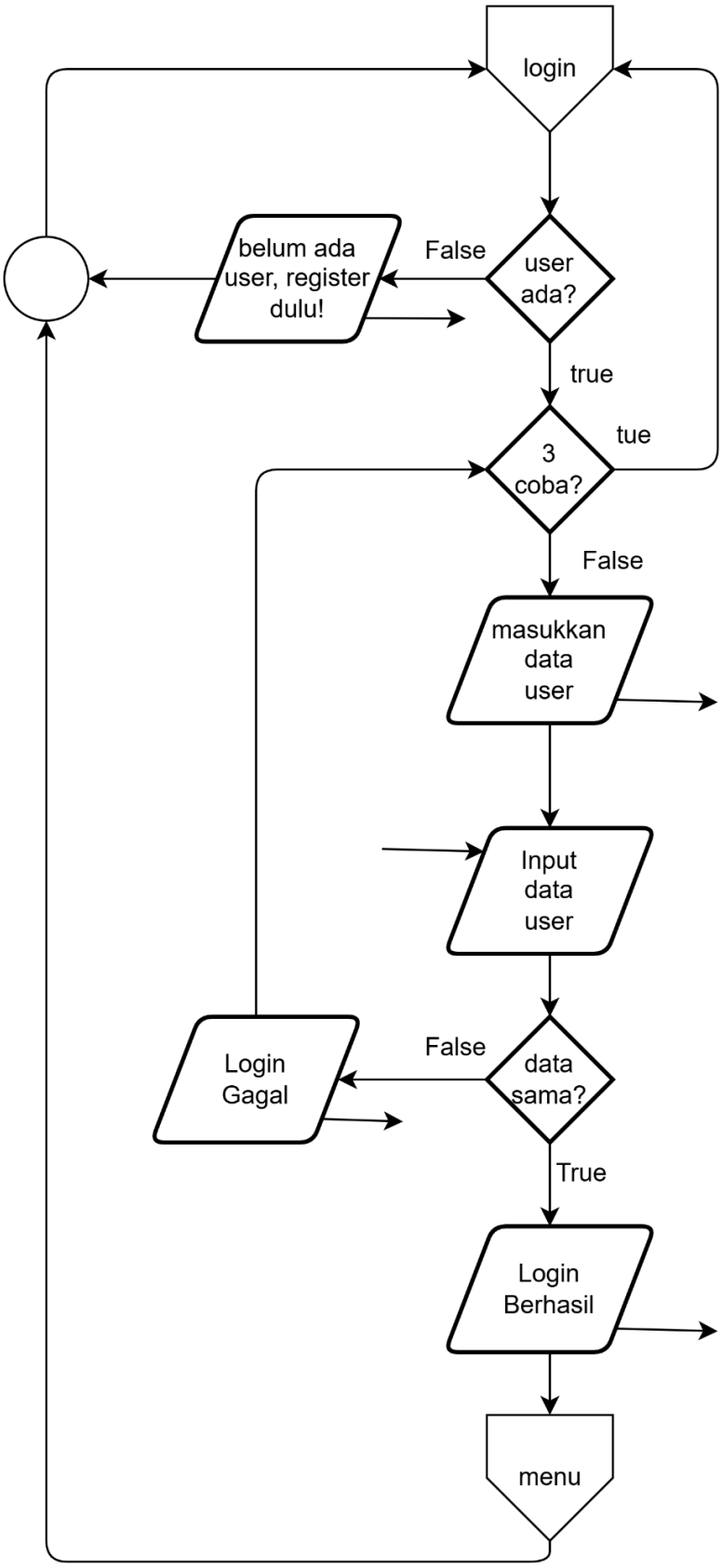
Gambar 1.1

Pada **Gambar 1.1**, diagram alir menjelaskan alur logika awal sistem yang dimulai dengan inisialisasi (Start) dan dilanjutkan dengan penampilan menu autentikasi. Setelah pengguna memberikan input pilihan autentikasi, sistem akan melakukan percabangan kondisi berdasarkan angka yang dimasukkan. Pilihan 1 diarahkan ke proses registrasi ('register'), sedangkan pilihan 2 diarahkan ke proses masuk ('login'), dan pilihan 3 akan mengarah ke terminasi program ('end'). Diagram ini juga mencakup penanganan kesalahan input, di mana jika pilihan yang dimasukkan tidak sesuai dengan opsi yang tersedia, sistem akan menampilkan pesan "pilihan tidak tersedia".



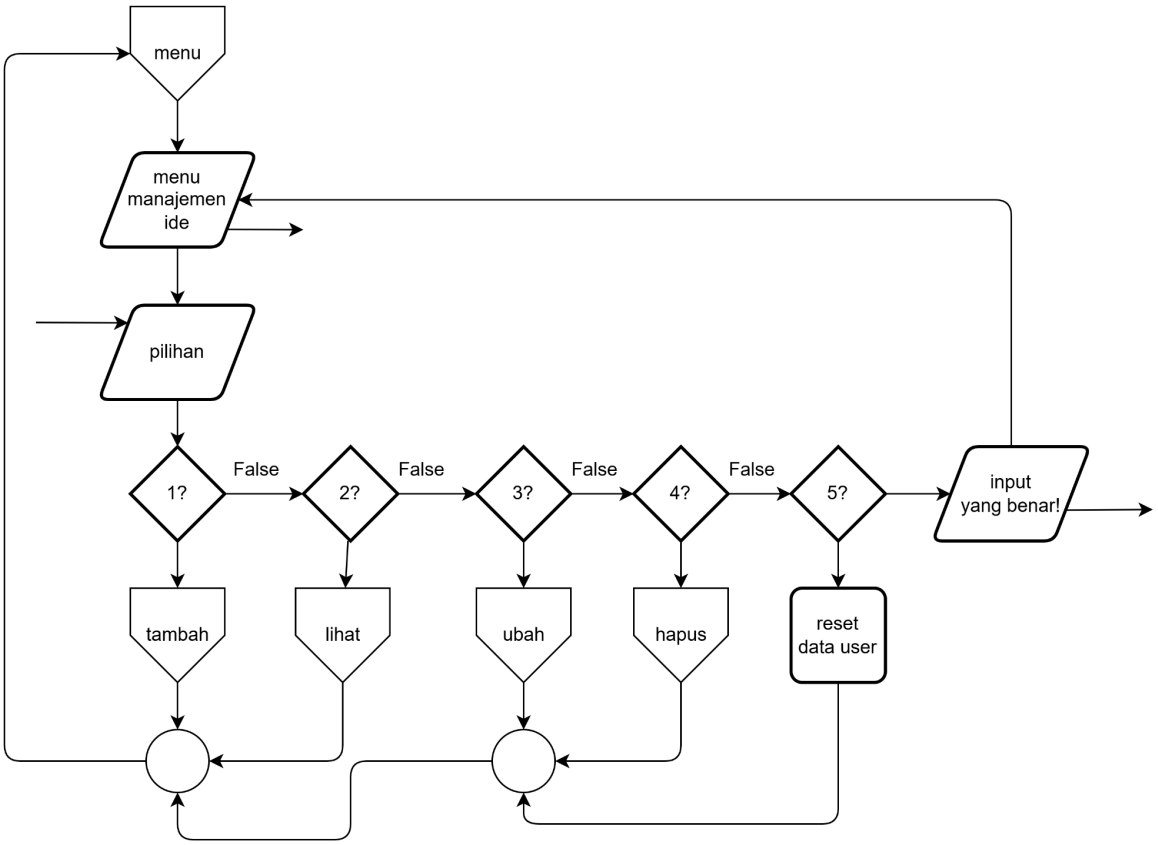
Gambar 1.2

Pada **Gambar 1.2**, diagram alir menguraikan logika prosedur untuk fitur registrasi atau pendaftaran akun baru. Alur dimulai dari konektor "register" dan segera melakukan validasi kapasitas penyimpanan melalui keputusan "user Penuh?". Apabila kondisi tersebut terpenuhi (ya), sistem akan langsung menampilkan pesan peringatan "User Penuh!". Sebaliknya, jika kapasitas masih tersedia (tidak), proses berlanjut ke tahap masukan di mana sistem menampilkan instruksi "masukkan data user", dilanjutkan dengan penerimaan "input data user" dari pengguna, dan diakhiri dengan pesan konfirmasi "register berhasil" yang menandakan selesainya proses pendaftaran.


login
belum ada user, register dulu!
False
user ada?
true
3 coba?
tue
False
masukkan data user
Input data user
Login Gagal
False
data sama?
True
Login Berhasil
menu

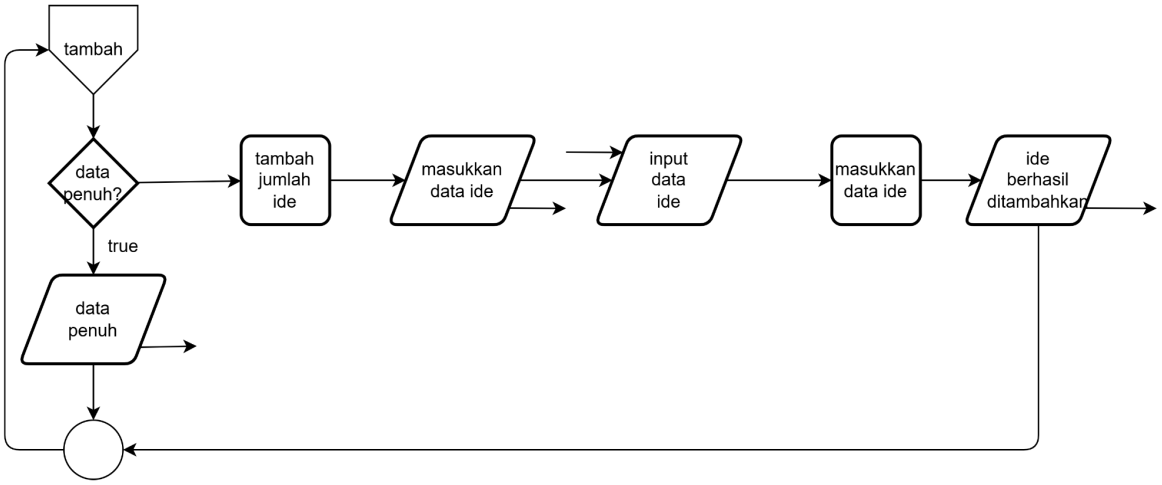
Gambar 1.3

Pada **Gambar 1.3**, diagram alir menguraikan logika prosedur untuk fitur masuk atau *login*. Proses diawali dengan validasi awal untuk memeriksa apakah terdapat data pengguna ("user ada?"); jika tidak, sistem akan menampilkan peringatan "belum ada user, register dulu!". Apabila data tersedia, alur berlanjut ke proses input data pengguna dengan mempertimbangkan batas percobaan ("3 coba?"). Setelah data dimasukkan, sistem melakukan pengecekan kesesuaian kredensial ("data sama?"); jika data tidak sesuai, sistem menampilkan pesan "Login Gagal", namun jika data valid, sistem akan memberikan notifikasi "Login Berhasil" dan mengarahkan pengguna ke menu utama ("menu").



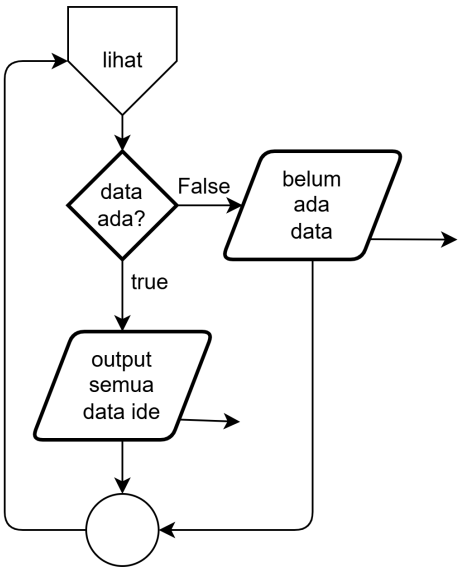
Gambar 1.4

Pada **Gambar 1.4**, diagram alir menjelaskan alur navigasi pada menu manajemen ide yang diakses setelah pengguna berhasil masuk ke sistem. Proses dimulai dengan menampilkan daftar opsi fitur ("menu manajemen ide") dan menerima input pilihan dari pengguna. Berdasarkan input tersebut, sistem melakukan percabangan kondisi untuk mengeksekusi fungsi yang sesuai: pilihan 1 mengarah ke proses "tambah" data, pilihan 2 ke proses "lihat" data, pilihan 3 ke proses "ubah" data, pilihan 4 ke proses "hapus" data, dan pilihan 5 berfungsi untuk melakukan "reset data user". Selain itu, diagram ini juga mencakup penanganan kesalahan input, di mana jika pilihan yang dimasukkan tidak valid, sistem akan menampilkan pesan "input yang benar!" untuk mengarahkan pengguna memilih opsi yang tersedia.



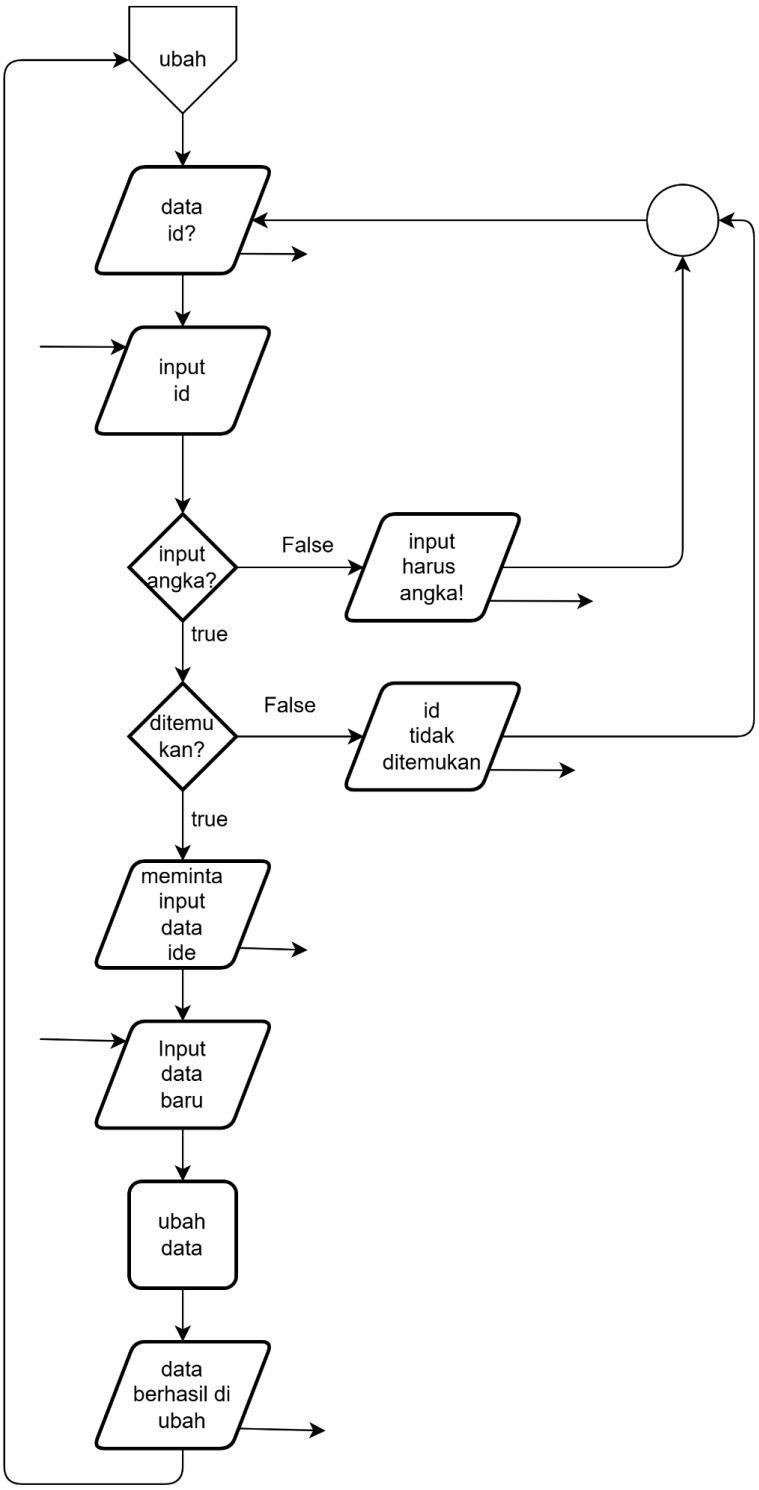
Gambar 1.5

Pada **Gambar 1.5**, diagram alir menjelaskan logika prosedur untuk fitur penambahan data ide. Proses dimulai dari konektor "tambah" dan segera melakukan validasi kapasitas penyimpanan melalui keputusan "data penuh?"; jika kondisi ini terpenuhi, sistem akan menampilkan pesan "data penuh". Sebaliknya, jika kapasitas masih tersedia, sistem akan melakukan proses "tambah jumlah ide" untuk memperbarui penghitung, dilanjutkan dengan menampilkan instruksi "masukkan data ide", menerima "input data ide" dari pengguna, memproses penyimpanan data tersebut, dan diakhiri dengan pesan konfirmasi bahwa "ide berhasil ditambahkan".



Gambar 1.6

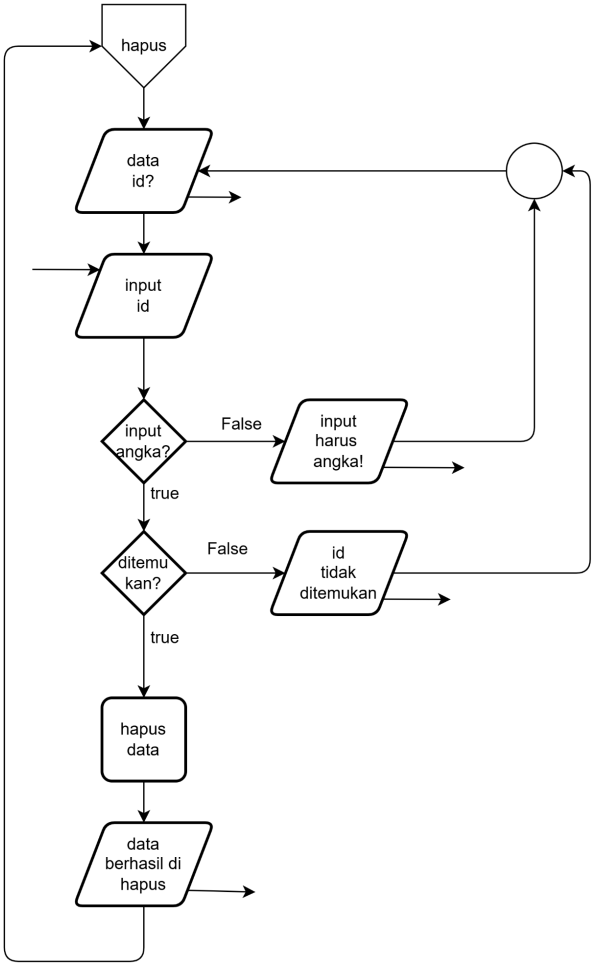
Pada **Gambar 1.6**, diagram alir menguraikan logika prosedur untuk fitur melihat atau menampilkan data ("lihat"). Proses diawali dengan validasi ketersediaan data melalui keputusan "data ada?"; apabila tidak terdapat data, sistem akan menampilkan notifikasi "belum ada data". Namun, jika data tersedia, sistem akan melanjutkan proses dengan menampilkan seluruh informasi yang tersimpan melalui "output semua data ide", sehingga pengguna dapat melihat daftar ide yang telah dimasukkan ke dalam sistem.



Gambar 1.7

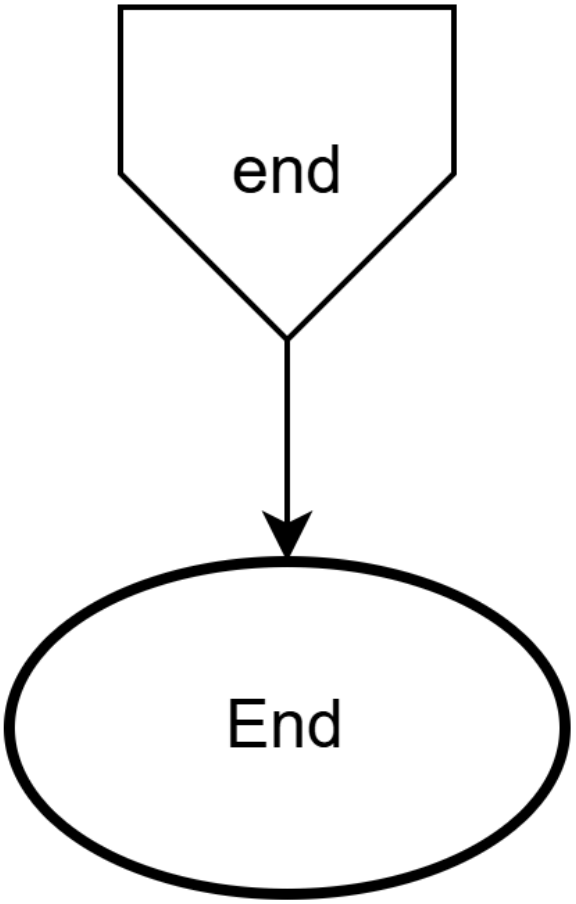
Pada **Gambar 1.7**, diagram alir menjelaskan prosedur logika untuk fitur pembaruan atau pengubahan data ("ubah"). Proses dimulai dengan meminta pengguna memasukkan identitas data melalui "input id". Sistem kemudian melakukan validasi bertingkat, pertama dengan memastikan apakah input tersebut berupa angka ("input angka?"); jika bukan, sistem akan menampilkan peringatan "input harus angka!". Jika valid, sistem akan mengecek keberadaan data tersebut ("ditemukan?"); apabila ID tidak ditemukan, akan muncul notifikasi

"id tidak ditemukan". Namun, jika ID ditemukan, sistem akan meminta pengguna untuk memasukkan data ide baru ("Input data baru"), memproses perubahan tersebut ("ubah data"), dan diakhiri dengan pesan konfirmasi bahwa "data berhasil di ubah".



Gambar 1.8

Pada **Gambar 1.8**, diagram alir menguraikan logika prosedur untuk fitur penghapusan data ("hapus"). Proses diawali dengan permintaan identitas data melalui output "data id?" dan penerimaan "input id" dari pengguna. Selanjutnya, sistem melakukan validasi input dengan mengecek apakah masukan tersebut berupa angka ("input angka?"); jika tidak, sistem menampilkan peringatan "input harus angka!". Apabila input valid, sistem melanjutkan pengecekan keberadaan data ("ditemukan?"); jika ID tidak ditemukan, akan muncul notifikasi "id tidak ditemukan". Namun, jika ID ditemukan, sistem akan menjalankan proses "hapus data" dan diakhiri dengan pesan konfirmasi bahwa "data berhasil di hapus".



Gambar 1.9

Pada **Gambar 1.9**, diagram alir menggambarkan tahap terminasi atau penghentian program yang menandakan akhir dari seluruh eksekusi sistem. Flowchart ini menunjukkan konektor "end" yang mengarah langsung ke simbol terminator "End", yang secara visual merepresentasikan titik akhir dari alur logika program. Simbol ini berfungsi sebagai penanda bahwa seluruh rangkaian proses dalam aplikasi telah selesai dijalankan dan program akan berhenti beroperasi secara resmi.

2. Deskripsi Singkat Program

Program ini merupakan aplikasi berbasis console untuk **Manajemen Ide** yang dibangun menggunakan bahasa C++. Sistem ini mengintegrasikan fitur **autentikasi** (Register dan Login) dengan batasan kapasitas maksimal 50 pengguna dan 3 kali percobaan login. Setelah berhasil masuk, pengguna dapat melakukan operasi **CRUD** (Tambah, Lihat, Ubah, Hapus) pada data ide (maksimal 100 data) yang mencakup atribut judul, deskripsi, dan status. Program ini juga dilengkapi dengan validasi input yang ketat untuk mencegah error saat pengguna memasukkan data yang tidak sesuai.

3. Source Code

A. Fitur Register

Fitur ini digunakan untuk mendaftarkan pengguna baru ke dalam sistem. Program akan menyimpan nama dan NIM pengguna ke dalam array daftarUser. Sistem memiliki batasan maksimal 50 pengguna.

Source Code:

```
if(pilihanAutentikasi == 1){
    if(sistem.jumlahUser >= 50){
        cout << "User sudah penuh" << endl;
    }
    else{
        cout << "Nama: ";
        cin >> sistem.daftarUser[sistem.jumlahUser].nama;
        cout << "NIM: ";
        cin >> sistem.daftarUser[sistem.jumlahUser].nim;
        sistem.jumlahUser++;
        cout << "Register berhasil" << endl;
    }
}
```

B. Fitur Login

Fitur ini digunakan untuk autentikasi pengguna yang sudah terdaftar. Sistem memberikan 3 kali percobaan untuk memasukkan nama dan NIM yang sesuai dengan data yang tersimpan.

Source Code:

```
else if(pilihanAutentikasi == 2){
    if(sistem.jumlahUser == 0){
        cout << "Belum ada user, silakan register dulu" << endl;
    }
    else{
        string namaLogin, nimLogin;
        int percobaan = 0;
```

```
        bool login = false;

        while(percobaan < 3){
            cout << "Nama: ";
            cin >> namaLogin;
            cout << "NIM: ";
            cin >> nimLogin;

            for(int i=0;i<sistem.jumlahUser;i++){
                if(namaLogin == sistem.daftarUser[i].nama &&
                  nimLogin == sistem.daftarUser[i].nim){
                   login = true;
                   break;
                }
            }

            if(login){
                cout << "Login berhasil" << endl;
                break;
            }
            else{
                cout << "Login gagal" << endl;
                percobaan++;
            }
        }
    }
}
```

C. Fitur Tambah Ide

Fitur ini memungkinkan pengguna yang sudah login untuk menambahkan ide baru dengan atribut judul, deskripsi, dan status. Sistem memiliki batasan maksimal 100 ide dan ID dibuat otomatis.

Source Code:

```
if(menu == 1){
    if(sistem.jumlahIde >= 100){
        cout << "Data penuh" << endl;
    }
    else{
        cin.ignore();
        sistem.daftarIde[sistem.jumlahIde].id = sistem.jumlahIde + 1;
```

```
        cout << "Judul Ide: ";
        getline(cin,sistem.daftarIde[sistem.jumlahIde].judul);

        cout << "Deskripsi Ide: ";
        getline(cin,sistem.daftarIde[sistem.jumlahIde].deskripsi);

        // Pilihan status
        cout << "Status Ide" << endl;
        cout << "1 Direncanakan" << endl;
        cout << "2 Ditunda" << endl;
        cout << "3 Berjalan" << endl;
        cout << "4 Dibatalkan" << endl;
        cout << "5 Tidak Jadi" << endl;

        while(true){
            cout << "Pilih: ";
            cin >> pilihanStatus;
            if(cin.fail()){
                cin.clear();
                cin.ignore(1000,'\n');
                cout << "Input harus angka" << endl;
                continue;
            }
            if(pilihanStatus>=1 && pilihanStatus<=5)
                break;
            cout << "Pilihan tidak tersedia" << endl;
        }

        // Set status berdasarkan pilihan
        if(pilihanStatus==1) sistem.daftarIde[sistem.jumlahIde].status="Direncanakan";
        if(pilihanStatus==2) sistem.daftarIde[sistem.jumlahIde].status="Ditunda";
        if(pilihanStatus==3) sistem.daftarIde[sistem.jumlahIde].status="Berjalan";
        if(pilihanStatus==4) sistem.daftarIde[sistem.jumlahIde].status="Dibatalkan";
        if(pilihanStatus==5) sistem.daftarIde[sistem.jumlahIde].status="Tidak Jadi";

        sistem.jumlahIde++;
        cout << "Ide berhasil ditambahkan" << endl;
    }
}
```

D. Fitur Lihat Ide

Fitur ini digunakan untuk menampilkan semua ide yang tersimpan dalam sistem dengan format tabel yang rapi menggunakan setw untuk pengaturan lebar kolom.

Source Code:

```
else if(menu == 2){
    if(sistem.jumlahIde == 0){
        cout << "Belum ada data" << endl;
    }
    else{
        cout << left
            << setw(6) << "ID"
            << setw(25) << "Judul"
            << setw(40) << "Deskripsi"
            << setw(15) << "Status" << endl;

        cout << string(86, '-') << endl;

        for(int i=0;i<sistem.jumlahIde;i++){
            cout << left
                << setw(6) << sistem.daftarIde[i].id
                << setw(25) << sistem.daftarIde[i].judul
                << setw(40) << sistem.daftarIde[i].deskripsi
                << setw(15) << sistem.daftarIde[i].status
                << endl;
        }
    }
}
```

E. Fitur Ubah ide

Fitur ini memungkinkan pengguna untuk mengubah data ide yang sudah ada berdasarkan ID. Pengguna dapat mengubah judul, deskripsi, dan status ide.

Source Code :

```
else if(menu == 3){
    int id;
    bool ditemukan = false;

    cout << "Masukkan ID: ";
    while(!(cin >> id)){
        cin.clear();
        cin.ignore(1000,'\n');
        cout << "Input harus angka: ";
    }

    for(int i=0;i<sistem.jumlahIde;i++){
```

```
        if(sistem.daftarIde[i].id == id){
            ditemukan = true;
            cin.ignore();

            cout << "Judul baru: ";
            getline(cin,sistem.daftarIde[i].judul);

            cout << "Deskripsi baru: ";
            getline(cin,sistem.daftarIde[i].deskripsi);

            // Update status (sama seperti tambah)
            // ... kode pilihan status ...

            cout << "Data berhasil diubah" << endl;
            break;
        }
    }

    if(!ditemukan){
        cout << "ID tidak ditemukan" << endl;
    }
}
```

F. Fitur Hapus Ide

Fitur ini digunakan untuk menghapus data ide berdasarkan ID. Setelah data dihapus, seluruh data setelahnya akan bergeser untuk menutupi posisi data yang dihapus.

```
else if(menu == 4){
    int id;
    bool ditemukan = false;

    cout << "Masukkan ID: ";
    while(!(cin >> id)){
        cin.clear();
        cin.ignore(1000,'\n');
        cout << "Input harus angka: ";
    }

    for(int i=0;i<sistem.jumlahIde;i++){
        if(sistem.daftarIde[i].id == id){
            // Geser data
            for(int j=i;j<sistem.jumlahIde-1;j++){
```

```
                sistem.daftarIde[j] = sistem.daftarIde[j+1];
            }

            sistem.jumlahIde--;
            ditemukan = true;
            cout << "Data berhasil dihapus" << endl;
            break;
        }
    }

    if(!ditemukan){
        cout << "ID tidak ditemukan" << endl;
    }
}
```

4. Hasil Output

```
===== MENU AWAL =====
1 Register
2 Login
3 Keluar Program
Pilihan: 1
Nama: zidan
NIM: 021
Register berhasil
===== MENU AWAL =====
1 Register
2 Login
3 Keluar Program
Pilihan: 2
Nama: zidan
NIM: 021
Login berhasil

===== MENU Manajemen Ide =====
1 Tambah Ide
2 Lihat Ide
3 Ubah Ide
4 Hapus Ide
5 Logout
Pilihan:
```

5. Langkah-langkah GIT

5.1 GIT Add

```
PS D:\kuliah\semester2\praktikum\apl\praktikum-apl> git add .
```

menambahkan perubahan pada repositori

5.2 GIT Commit

```
PS D:\kuliah\semester2\praktikum\apl\praktikum-apl> git commit -m "PT-2"
[main c8c7fbf] PT-2
 3 files changed, 967 insertions(+), 9 deletions(-)
 create mode 100644 flowchart/PT-2.drawio
 create mode 100644 post-test/post-test-apl-2/2509106021-MuhammadZidaneAbdulKadir-PT-2.exe
PS D:\kuliah\semester2\praktikum\apl\praktikum-apl>
```

melakukan commit pada repositori

5.3 GIT Push

```
PS D:\kuliah\semester2\praktikum\apl\praktikum-apl> git push
Enumerating objects: 13, done.
Counting objects: 100% (13/13), done.
Delta compression using up to 16 threads
Compressing objects: 100% (8/8), done.
Writing objects: 100% (8/8), 28.85 KiB | 2.22 MiB/s, done.
Total 8 (delta 1), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (1/1), completed with 1 local object.
To github.com:ExtraYiban/praktikum-apl.git
   c37df2f..c8c7fbf  main -> main
```

mengiriim commit ke github